Quality Architecture for AI Coding Agent Harnesses

First Author[1], Second Author[2], Third Author[2], Fourth Author[1] and Fifth Author[1]
[*]Equal Contribution, [1]Affiliation 1, [2]Affiliation 2

AI code generators now produce an estimated 40–42% of production code, yet observational studies consistently report 1.7× more issues, 2.7× more security vulnerabilities, and a collapse of refactoring activity from 25% to under 10% of changed lines. The resulting defects ("slop") are distinctive: they are superficially competent, individually defensible, and cumulatively corrosive to codebase health.
We present a formal framework for reasoning about layered defense against AI code slop. Our contributions are: (1) a formal taxonomy of 18 slop types classified by detection decidability into Decidable, Semi-decidable, and Undecidable classes, grounded in Rice's theorem; (2) the *Layered Defense Optimization Problem*, proved NP-hard via reduction from Weighted Set Cover, with a $(1 - 1/e)$-approximate greedy algorithm; (3) the *Detection Ceiling Theorem*, using the Data Processing Inequality to prove hard upper bounds on layer-specific detection rates; (4) the *Correlated Judge-Producer Error Model*, formalizing why same-model LLM-as-Judge systems underperform the independence assumption via the FKG inequality; (5) the *Compound Degradation Theorem*, proving that individually CI-passing changes can produce superlinear entropy growth; and (6) the *Accretion Category*, a novel defect class absent from all classical taxonomies (IEEE 1044, ODC, Beizer, CWE) that captures code which is correct, unnecessary, individually defensible, and collectively harmful.
We calibrate these formal results against empirical data from eight large-scale studies (CodeRabbit, GitClear, Spotify Honk, METR, Apollo Research, AgentSpec, Veracode, NIST CAISI) and derive a four-layer quality stack ordered by cost and determinism. The framework provides closed-form expressions for optimal quality investment per code category and identifies the four slop types (meaningless tests, boilerplate inflation, redundant comments, naming drift) that constitute the detection research frontier.

1. Introduction

For the first time in recorded software history, developers paste code more often than they refactor or reuse it (GitClear, 2025). AI code generators have crossed the threshold from occasional suggestion to primary author: converging estimates place AI-generated code at 40–42% of production output (GitClear, 2025; Sonar, 2026). This shift has produced a new category of software quality problem.

The problem is not that AI writes bad code occasionally. The problem is that it writes *superficially competent* code at scale: code that passes type-checkers, satisfies linters, and clears CI pipelines, yet cumulatively degrades codebase health through unnecessary abstractions, duplicated logic, happy-path-only error handling, and silent scope expansion. We call these defects *slop*, following industry usage.

The empirical evidence is stark. CodeRabbit's analysis of 470 pull requests found 1.68× more issues in AI-generated PRs, with 2.74× more XSS vulnerabilities and approximately 8× more I/O problems (CodeRabbit, 2025). GitClear's analysis of 211 million lines found refactoring collapsed from 24.1% to 9.5% of changes while code cloning grew 8× (GitClear, 2025). A study of 304,362 AI commits across 6,275 repositories found 484,606 issues, with security issues surviving at 41.1% (Siddiq et al., 2026). Veracode tested 100+ LLMs on 80 security-sensitive tasks and found a 45% failure rate, with zero improvement over time (Veracode, 2025).

Yet a controlled trial by GitHub (202 developers, single task) found quality *improvements* of 2.5–4.2% (Peng et al., 2024). This contradiction between controlled and observational studies is itself informative: AI *can* produce high-quality code under narrow, supervised conditions, but *does not* in aggregate practice. The verification bottleneck (only 48% of developers always verify AI output (Sonar, 2026)) is the likely mechanism.

This paper provides a formal framework for reasoning about what to verify, how aggressively, and with what tools. We make the following contributions:

1. A **formal slop taxonomy** (§3) classifying 18 defect types into three decidability classes (Decidable, Semi-decidable, Undecidable), with three approximately orthogonal generative factors (Context Blindness, Completion Bias, Training Distribution Leakage).
2. The **Layered Defense Optimization Problem** (§4), proved NP-hard, with a greedy $(1 - 1/e)$-approximation via submodularity.
3. The **Detection Ceiling Theorem** (§5), proving information-theoretic upper bounds on per-layer detection rates using the Data Processing Inequality.
4. The **Correlated Judge-Producer Error Model** (§6), showing that shared training data creates positively correlated failures between producer and judge LLMs.
5. The **Compound Degradation Theorem** (§9), proving superlinear entropy growth from individually CI-passing changes, with the *Accretion Category* as a novel defect class.
6. **Empirical calibration** (§11) of all formal model parameters against eight large-scale studies, yielding a cost-optimal four-layer quality stack with per-type layer assignments.

Scope and framing. This is a theory and position paper. Our novel contributions are in the *application* of established formal machinery (information theory, decision theory, computability theory) to the new problem of AI code quality defense, and in the *synthesis* of empirical data into calibrated formal models. The individual theorems draw on classical results; the contribution is their composition into a coherent framework for an emerging problem.

2. Preliminaries and Definitions

2.1. Notation

Let $J = \{1, \ldots, 18\}$ index the slop types. Let $L = \{1, 2, 3, 4\}$ index defense layers: structural gates, deterministic analysis, LLM-as-Judge, and human review. For each layer $\ell \in L$ and defect type $j \in J$:

- $d_{\ell,j} \in [0, 1]$: detection probability (sensitivity) of layer ℓ for type j
- $c_\ell > 0$: cost of applying layer ℓ per code unit
- $D_j > 0$: downstream cost of an undetected defect of type j
- $p_j \in [0, 1]$: prior probability that a code unit contains defect type j
- $\rho_{\ell,\ell',j} \in [-1, 1]$: correlation between detection failures of layers ℓ and ℓ'

Let $\Lambda_j \subseteq L$ denote the subset of layers applied to defect type j.

2.2. The Four Defense Layers

The quality stack is ordered by cost and determinism:

Layer 1: Structural gates (~2s per write). Type-checkers, import validators, lint with zero-suppression policy, diff budget enforcement, dead code detection. Deterministic, cheap, applied to every write.

Layer 2: Deterministic analysis (~10s per task). AST scan for stubs/placeholders, clone detection, cyclomatic complexity thresholds, deprecation database checks. Deterministic but requires tooling setup.

Layer 3: LLM-as-Judge (~30s per task). Scope compliance, abstraction audit, test quality review, naming consistency. Probabilistic, subject to judge-model correlation.

Layer 4: Human review (flagged items only). Architectural coherence, security review for high-risk paths, design intent validation. Highest quality but resource-constrained.

Layers 1–2 block commits. Layer 3 flags but does not block. Layer 4 sees only what survived three automated layers.

3. Formal Slop Taxonomy

3.1. Defect Type as a Formal Object

Definition 3.1 (Slop Type)**.** *A slop type is a tuple $\tau_j = (\sigma_j, \phi_j, \delta_j, C_j)$ where:*

- $\sigma_j : \mathrm{AST} \to \{0,1\}$ *is the* ***syntactic signature****, a predicate on abstract syntax trees identifying candidate instances.*
- $\phi_j : \mathrm{Program} \times \mathrm{Context} \to \{0,1\}$ *is the* ***semantic property****, determining whether a candidate is a genuine defect given full program context.*
- $\delta_j \in \{\mathrm{D}, \mathrm{SD}, \mathrm{U}\}$ *is the* ***detection decidability class****.*
- $C_j : \Omega \to \mathbb{R}_{\geq 0}$ *is the* ***cost distribution*** *over deployment contexts.*

Definition 3.2 (Detection Decidability Classes)**.** *The three classes are formally distinct:*

- ***Decidable (D):*** *There exists a total computable function f_j such that $f_j(t) = 1$ iff t contains an instance of τ_j. The syntactic signature σ_j suffices; the semantic property ϕ_j does not depend on unobservable context.*
- ***Semi-decidable (SD):*** *The semantic property ϕ_j depends on partially formalizable context. An oracle (LLM or human) can approximate ϕ_j with probability $p \in (0.5, 1)$, but no computable function achieves $p = 1$.*
- ***Undecidable (U):*** *No known oracle achieves reliable detection. For all known procedures f, the inter-rater reliability $\kappa < 0.4$ (below "fair agreement").*

3.2. Partition of the 18 Types

Proposition 3.1 (Decidability Partition)**.** *The 18 slop types partition as shown in Table 1.*

Proof sketch. For each Decidable type, we exhibit the computable function: hallucinated imports are detected by lockfile resolution (table lookup, $O(1)$); placeholder code by AST pattern matching for pass, ..., $\mathrm{NotImplementedError}$ (regular language over AST tokens); duplicate logic by tree-sitter-based clone detection (suffix tree, $O(n \log n)$); dead code by reachability analysis on the call graph;

Table 1 | The 18 slop types partitioned by detection decidability.

Class	Count	Types
D (Decidable)	7	Hallucinated imports, placeholder/stub code, duplicate logic, dead code, lint suppressions, cross-language contamination, outdated APIs
SD (Semi-decidable)	7	Security vulns (functional-looking), happy-path error handling, data model mismatches, over-engineering, architectural erosion, silent scope expansion, god functions
U (Undecidable)	4	Meaningless tests, boilerplate inflation, redundant comments, naming/convention drift

lint suppressions by regex; cross-language contamination by language-specific AST rules; outdated APIs by deprecation database lookup.

For Semi-decidable types, the semantic property ϕ_j depends on context: security vulnerabilities require knowing which inputs are adversarial; happy-path handling requires knowing which error paths matter; over-engineering requires judging abstraction necessity. Experts agree at $\kappa \approx 0.6$ (substantial agreement), confirming detectability above chance but below certainty.

For Undecidable types, even expert reviewers disagree: "meaningless test" depends on test intent versus structure (domain-dependent); "boilerplate" thresholds vary by team norms; "redundant comment" depends on reader expertise; "naming drift" requires inferring implicit conventions. Inter-rater $\kappa < 0.4$ for all four (Stureborg et al., 2024). □

3.3. Latent Factor Structure

We propose three approximately orthogonal generative factors:

Definition 3.3 (Context Blindness (F1))**.** *The agent's effective context $\hat{\mathcal{K}}(t) \subset \mathcal{K}(t)$, where $\mathcal{K}(t)$ is the codebase knowledge required for correct generation at point t. The probability of slop type τ_j increases monotonically with the context deficit $|\mathcal{K} \setminus \hat{\mathcal{K}}|$. Types loading on F1: duplicate logic, naming drift, architectural erosion, boilerplate inflation.*

Definition 3.4 (Completion Bias (F2))**.** *The agent's output distribution π assigns higher probability to syntactically complete outputs than to semantically minimal correct outputs: $\mathbb{E}_{\pi}[\text{length}(o)] > \mathbb{E}_{\pi^*}[\text{length}(o)]$ where π^* is optimal. Types loading on F2: placeholder code, happy-path error handling, meaningless tests, redundant comments, over-engineering.*

Definition 3.5 (Training Distribution Leakage (F3))**.** *Patterns memorized during training are applied to contexts where they do not hold. Probability increases with divergence between training distribution $\mathcal{D}$ and execution environment $\mathcal{E}$. Types loading on F3: hallucinated imports, cross-language contamination, outdated APIs, security vulnerabilities.*

Proposition 3.2 (Approximate Orthogonality (Hypothesis))**.** *We* hypothesize *that the three factors are approximately orthogonal: $I(F_i; F_k)$ is small relative to $H(F_i)$ for $i \neq k$, because they operate at different stages of the generation process (input retrieval, decoding objective, prior knowledge respectively). Confirmatory factor analysis on a labeled slop dataset (the ISSRE 2025 dataset with 500K+ samples is the most promising candidate) is needed to validate this structure.*

Corollary 3.1 (Intervention Independence)**.** *If the three factors are approximately orthogonal, interventions targeting different factors compose multiplicatively. Better retrieval (addressing F1) does not reduce completion bias (F2), and vice versa. The quality stack must address all three independently.*

3.4. Relationship to Classical Taxonomies

No classical taxonomy captures the full slop landscape. IEEE 1044 (iee, 2009) assumes the failure-defect-fault chain, but slop often causes no failures. ODC (Chillarege et al., 1992) classifies defects requiring fixes, but individually acceptable slop does not require fixes. Beizer (1990) classifies bugs traceable to incorrect decisions, but no specific decision in an accretion defect is incorrect. CWE classifies security weaknesses, covering only type 8. Four types (over-engineering, architectural erosion, scope expansion, boilerplate inflation) map poorly to every classical taxonomy because they presuppose intentional authorship.

4. The Layered Defense Optimization Problem

4.1. Problem Statement

For each defect type j, select $\Lambda_j \subseteq L$. The escape probability under correlation is:

$$P_{\text{esc}}(j, \Lambda_j) = P\left(\bigcap_{\ell \in \Lambda_j} \{Z_{\ell,j} = 0\}\right) \tag{1}$$

where $Z_{\ell,j} \in \{0, 1\}$ indicates detection by layer ℓ. Under independence, $P^{\text{ind}}_{\text{esc}}(j, \Lambda_j) = \prod_{\ell \in \Lambda_j}(1 - d_{\ell,j})$. Under pairwise correlation ρ between layers ℓ_1, ℓ_2:

$$P(Z_{\ell_1} = 0, Z_{\ell_2} = 0) = (1 - d_1)(1 - d_2) + \rho\sqrt{d_1(1 - d_1) \cdot d_2(1 - d_2)} \tag{2}$$

With shared layer costs (a layer activated for any type is paid once):

$$C_{\text{total}} = \sum_{\ell \in L} c_\ell \cdot 1[\Lambda^\ell \neq \emptyset] + \sum_{j \in J} p_j \cdot D_j \cdot P_{\text{esc}}(j, \Lambda_j) \tag{3}$$

where $\Lambda^\ell = \{j : \ell \in \Lambda_j\}$.

Theorem 4.1 (Hardness)**.** *The Layered Defense Optimization Problem with shared layer costs is NP-hard, by reduction from Weighted Set Cover.*

Proof sketch. Set $D_j = M$ for large M and $p_j = 1$ for all j. Then minimizing the objective forces all types to be covered (the penalty $M \cdot P_{\text{esc}} \gg c_\ell$ for any uncovered type), reducing to minimum-weight subcollection selection, which is Weighted Set Cover. □

Corollary 4.1. *Unless P = NP, no polynomial-time algorithm achieves approximation ratio better than* $\ln |J|$ *(Dinur and Steurer, 2014).*

Theorem 4.2 (Greedy Approximation)**.** *Under independence, the greedy algorithm (iteratively select the layer with highest marginal ROI* Δ_ℓ / c_ℓ*) achieves a* $(1 - 1/e)$*-approximation to the optimal escaped defect cost reduction.*

Proof sketch. Under independence, the marginal escaped-defect-cost reduction from adding a layer is monotone submodular (adding a layer never increases escape probability, and marginal reduction diminishes as more layers are selected). The result follows from Nemhauser et al. (1978). □

Proposition 4.1 (Decomposition)**.** *When costs are per-invocation (not shared) and layers are independent, the optimization decomposes into* $|J|$ *independent per-type subproblems, each solvable by ROI ordering.*

Remark. *For our problem (*$|L| = 4$*,* $|J| = 18$*), exact enumeration over* $2^4 = 16$ *subsets per type is computationally trivial. The greedy bound matters for scaled versions with dozens of specialized tools.*

5. Detection Ceiling Theorem

Let $X \in \{0, 1\}$ be the defect state and C the full context. Each layer ℓ observes $Y_\ell = f_\ell(C)$, forming the Markov chain $X \to C \to Y_\ell \to Z_\ell$.

Theorem 5.1 (Detection Ceiling). *For any layer ℓ with observables Y_ℓ:*

$$I(X; Z_\ell) \leq I(X; Y_\ell) \leq I(X; C) \tag{4}$$

Consequently, by Fano's inequality, the detection error probability satisfies:

$$h(P_e^{(\ell)}) \geq H(X|Y_\ell) = H(X) - I(X; Y_\ell) \tag{5}$$

where $h(\cdot)$ is the binary entropy function.

Proof. By the Data Processing Inequality (Cover and Thomas, 2006), mutual information can only decrease along a Markov chain. Fano's inequality lower-bounds error probability in terms of conditional entropy. □

Table 2 | Observable context and detection ceilings per layer.

Layer	Observable Y_ℓ	Excluded context
L1: Structural	Syntax, types, imports, diff size	Semantics, intent, requirements, runtime
L2: Deterministic	AST, clones, complexity, deprecation DB	Intent, requirements, runtime
L3: LLM-Judge	Code + task description + partial repo	Full runtime, complete requirements
L4: Human	All above + domain expertise + tacit knowledge	Exhaustive analysis (bounded by attention)

Theorem 5.2 (No Free Lunch for Context-Poor Layers). *For defect type j whose determination requires context C_j^*, if layer ℓ does not observe C_j^*, then $d_{\ell,j}$ is bounded well below the achievable rate with full context:*

$$I(X_j; Y_\ell) \leq I(X_j; C \setminus C_j^*) \tag{6}$$

Proof. Direct application of the DPI to $X_j \to C \to (Y_\ell, C_j^*) \to Y_\ell$, noting that dropping C_j^* can only reduce mutual information. □

Example. Silent scope expansion (type 13) requires $C_{13}^* = \{\text{task description}\}$. Layer 1 does not observe the task description, so $I(X_{13}; Y_1) \leq I(X_{13}; \text{code syntax alone}) \approx 0$, giving $d_{1,13} \approx 0$. This is confirmed empirically: structural gates catch 5–15% of scope expansion at best.

6. Correlated Judge-Producer Error Model

6.1. The Shared Blindspot Framework

Let M_P be the producer and M_J the judge. Define E_P as the event that M_P produces a defect and E_J as the event that M_J fails to detect it.

Proposition 6.1 (Correlated Blind Spots). *If both models learn similar representations from overlapping training data $\mathcal{T}_P \cap \mathcal{T}_J$, then defect patterns underrepresented in the shared data are both more likely to be produced and more likely to be missed, creating positive correlation:*

$$\text{Cov}(E_P, E_J) > 0 \tag{7}$$

Theorem 6.1 (Judge-Producer Correlation via FKG)**.** *Under a shared-difficulty model where both failure probabilities increase with difficulty θ (distance from training distribution), the FKG inequality (Fortuin et al., 1971) gives:*

$$E_\theta[\phi_P(\theta) \cdot \phi_J(\theta)] > E_\theta[\phi_P(\theta)] \cdot E_\theta[\phi_J(\theta)] \tag{8}$$

where $\phi_P(\theta)$ and $\phi_J(\theta)$ are the producer and judge failure probabilities at difficulty θ. The rate of escaped defects exceeds what the independence assumption predicts.

Proof. By the Harris-FKG inequality, for two non-decreasing functions on a distributive lattice, the expectation of the product exceeds the product of expectations. Both ϕ_P and ϕ_J are non-decreasing in difficulty θ. □

Corollary 6.1 (Independence Underestimates Escape)**.** *The standard formula $\prod_\ell (1 - d_{\ell,j})$ is a lower bound on true escape probability. Any cost analysis using independence underestimates escaped defect costs.*

6.2. Diversity Gain

Definition 6.1 (Diversity Gain)**.** *The gain from cross-model judging (using judge M_J^B from a different vendor than producer M_P^A) versus same-model judging (M_J^A) is:*

$$\Delta_{\text{div}} = P_{\text{esc}}^{A \text{ judges } A} - P_{\text{esc}}^{B \text{ judges } A} \tag{9}$$

This gain has two components: the detection rate difference (d_J^B versus d_J^A) and the correlation reduction ($\rho_{AB} < \rho_{AA}$). Even at equal detection rates, diversity gain is positive whenever $\rho_{AB} < \rho_{AA}$.

6.3. Connection to Littlewood-Strigini

Littlewood and Strigini (1993); Littlewood et al. (2000) proved that independently developed software versions do not fail independently, because some inputs are inherently difficult. Their result adapts directly: if producer and judge failure probabilities both increase with difficulty, then $P(\text{defect produced and missed}) > P(\text{produced}) \cdot P(\text{missed})$. Empirically, diverse techniques can achieve effectiveness *greater* than the independence assumption predicts (positive diversity gain), while same-technique repetitions underperform it (Littlewood et al., 2000).

7. Optimal Layer Ordering

Theorem 7.1 (ROI Ordering)**.** *Under independence, when layers form a cascade (resolved items skip subsequent layers), cost-optimal ordering sorts by decreasing r_ℓ / c_ℓ, where r_ℓ is the resolution rate.*

Proof sketch. Pairwise exchange argument (Smith's rule (Smith, 1956)): swapping adjacent layers ℓ_a, ℓ_b shows the ordering ℓ_a before ℓ_b is cheaper iff $r_a/c_a > r_b/c_b$. □

Proposition 7.1 (Correlation Breaks ROI Ordering)**.** *Under positive correlation, ROI ordering can be suboptimal. A concrete example: layers A ($d = 0.8$, $c = 10$, ROI $= 0.08D$) and B ($d = 0.6$, $c = 5$, ROI $= 0.12D$). Under independence, B first is optimal. But if a third layer C ($d = 0.5$, $c = 8$) has $\rho_{AC} = 0.1$ while $\rho_{BC} = 0.8$, then skipping B entirely and running A, C achieves lower total cost because A and C are nearly independent, yielding escape probability ~ 0.10 at cost 18 versus 0.36 at cost 23.*

This motivates diversity-aware layer selection: the marginal value of a layer depends more on its independence from existing layers than on its absolute detection rate.

8. The Turing Enforcement Principle

8.1. Rice's Theorem Applied to Enforcement

Theorem 8.1 (Rice, 1953)**.** *For any non-trivial semantic property P of programs (depending on input-output behavior, not syntactic form), the set $\{e : \phi_e \in P\}$ is undecidable.*

This motivates a three-way classification of enforcement:

Definition 8.1 (Enforcement Classes)**.**

- ***Syntactic properties*** *(decidable): depend only on program text. Enforceable with $P = 1$.*
- ***Quasi-semantic properties****: formally semantic but admit effective syntactic approximations for practical populations. Sound or complete but not both.*
- ***Semantic properties*** *(undecidable in general): depend on execution behavior. No enforcer achieves $P = 1$.*

Theorem 8.2 (Enforcement Gap)**.** *For any syntactic property P_S, there exists a deterministic enforcer with $P(\text{correct enforcement}) = 1$. For any semantic property P_U, every enforcer (including any LLM) satisfies $P(\text{correct enforcement}) < 1$. The gap cannot be closed by prompt engineering alone.*

Proof. Part 1: syntactic properties are decided by finite automata on the AST, achieving zero error probability.

Part 2: an LLM is a fixed-weight transformer mapping finite-length inputs to outputs. Its training distribution is finite, so for any non-trivial semantic property P_U, the space of programs exhibiting P_U is unbounded and cannot be fully represented in finite training data. Out-of-distribution inputs (novel programs not resembling training examples) will cause errors. More precisely: Rice's theorem establishes that no algorithm decides P_U for all programs; while an LLM is not a general Turing machine, the *practical consequence* is identical, since achieving $P = 1$ would require correctly classifying programs from the unbounded space of possible inputs, including those arbitrarily far from the training distribution. The error rate is bounded below by the probability mass on out-of-distribution inputs. □

The AgentSpec result (Wang et al., 2026) provides empirical calibration: 87.26% enforcement for code-based rules versus 77% for prompt-based rules. The 10-point gap is the measured enforcement gap. The METR and NIST CAISI findings (agents copying solutions, commenting out assertions, downloading answers) demonstrate the predicted failure mode: agents route around semantic intent while satisfying or subverting structural constraints (METR, 2025; NIST CAISI, 2026).

9. Compound Degradation and the Accretion Category

9.1. Codebase Entropy

Definition 9.1 (Codebase Entropy)**.** *For a codebase C of n files, let $\mu(f_i)$ be the minimum set of files a developer must understand to correctly modify f_i. The codebase entropy is:*

$$H(C) = \frac{1}{n} \sum_{i=1}^{n} \log_2 |\mu(f_i)| \tag{10}$$

A perfectly modular codebase has $H(C) = 0$; a fully coupled one has $H(C) = \log_2 n$.

Definition 9.2 (Accretion Rate)**.** *The accretion rate* $\alpha(t) = (H(C_{t+1}) - H(C_t))/|\Delta_t|$ *measures entropy increase per unit of code change.*

Theorem 9.1 (Compound Degradation)**.** *Let* $\{\Delta_t\}_{t=1}^{T}$ *be changes that each pass CI. If* $\alpha(t) > 0$ *for all* t *and modification context grows sublinearly (*$|\mu(f_i)| \sim n^{\beta}$*,* $0 < \beta < 1$*), then codebase entropy grows superlinearly in the number of changes.*

Proof sketch. Each change adds code without refactoring, increasing both n and coupling degree. The sum telescopes, and the $\log_2 |\mu(f_i)|$ term grows as $\beta \log_2 n$. CI checks are *pointwise* (verifying the current change) but entropy is *global* (depending on cumulative coupling). Pointwise correctness does not imply bounded global entropy. □

Remark. *The sublinear growth assumption (*$|\mu(f_i)| \sim n^{\beta}$*) is contingent on architecture. In well-modularized codebases with bounded-degree dependency graphs,* $|\mu(f_i)|$ *may be* $O(1)$*, yielding linear (not superlinear) entropy growth. The assumption is empirically plausible for AI-assisted development, where agents frequently create cross-module references without respecting module boundaries (GitClear's 8× clone increase is a proxy for coupling growth). Under* $O(1)$ *coupling, the theorem still guarantees monotonic entropy increase; the superlinearity depends on coupling growth outpacing modularization.*

Corollary 9.1 (The Ratchet Effect)**.** *Without refactoring proportional to the addition rate, entropy is monotonically non-decreasing. Each correct change ratchets entropy upward with no mechanism for spontaneous decrease.*

Corollary 9.2 (Refactoring Ratio Threshold)**.** *Let* r *be the fraction of refactoring changes (*$\alpha < 0$*) and* $(1 - r)$ *additions (*$\alpha > 0$*). Equilibrium requires* $r \cdot |\mathbb{E}[\alpha|\textit{refactoring}]| \geq (1 - r) \cdot \mathbb{E}[\alpha|\textit{addition}]$*. GitClear data indicates the pre-AI equilibrium at* $r \approx 0.24$*; the post-AI state at* $r \approx 0.095$ *violates the bound by approximately 2.5×.*

This connects to Lehman's Second Law (Lehman, 1980): as an E-type system evolves, its complexity increases unless explicit work is done to reduce it. AI-assisted development accelerates this by simultaneously increasing entropy-adding throughput, decreasing entropy-reducing refactoring, and introducing a class of entropy invisible to existing quality gates.

9.2. The Accretion Category

Definition 9.3 (Accretion Defect)**.** *A code change* Δ *is an accretion defect iff:*

1. ***Correctness:*** *all tests pass after* Δ*.*
2. ***Superfluity:*** *there exists* $\Delta' \subset \Delta$ *satisfying the same requirement.*
3. ***Individual defensibility:*** Δ *violates no stated standard.*
4. ***Aggregate harm:*** *cumulative application increases entropy by* $\epsilon > 0$ *per change.*

Proposition 9.1 (Taxonomic Gap)**.** *No classical taxonomy (IEEE 1044, ODC, Beizer, CWE) contains a category capturing accretion defects, because all four assume intentional authorship: code results from a human decision that was either correct or incorrect. Accretion arises from statistical pattern completion, which produces code because it is* probable*, not because it is* necessary*.*

Proposition 9.2 (Detection Asymmetry)**.** *Individual accretion detection is undecidable (requires exhibiting a smaller sufficient change, reducing to program equivalence). Aggregate detection is feasible via statistical process control on entropy proxies: accretion rate, refactoring ratio, clone frequency, lines per function point.*

Tragedy of the commons. Accretion has the formal structure of a commons tragedy (Hardin, 1968): each agent optimizes locally (task completion) without internalizing the externality (entropy increase). The quality stack can be interpreted as Pigouvian taxation (Layers 1–2: automated penalties for measurable entropy) plus Coasian property rights (Layer 4: code owner with veto power).

10. Defect Removal Efficiency Composition

Theorem 10.1 (DRE Cascade under Independence). *For L independent layers, cumulative DRE for type j is:*

$$D_j = 1 - \prod_{\ell=1}^{L}(1 - d_{\ell,j}) \tag{11}$$

Three layers at individually modest rates ($d_1 = 0.3$, $d_2 = 0.75$, $d_3 = 0.5$) yield $D = 1-(0.7)(0.25)(0.5) = 0.9125$, exceeding 91% cumulative DRE.

Theorem 10.2 (DRE under Gaussian Copula Correlation). *Under a Gaussian copula (Sklar, 1959) with correlation matrix* R*:*

$$D_j = 1 - C_{\mathrm{R}}(1 - d_{1,j}, \ldots, 1 - d_{L,j}) \tag{12}$$

At correlation $\rho = 1$, the second layer adds nothing ($D = d$). At $\rho = 0$, we recover independence.

Corollary 10.1 (Diversity Premium). *A weak independent detector ($d = 0.3, \rho = 0$) can outperform a strong correlated detector ($d = 0.7, \rho = 0.8$) in marginal DRE improvement.*

Proposition 10.1 (Capers Jones' 95% Law Explained). *Achieving DRE $> 95\%$ requires ≥ 3 diverse techniques (Jones and Bonsignour, 2012). With $d_{\max} \approx 0.85$ (best single technique) and realistic correlation $\rho \approx 0.3$, the copula model shows that removing any one of three layers ($d_1 = 0.60$, $d_2 = 0.50$, $d_3 = 0.70$) drops D below 0.90, while all three yield $D \approx 0.954$.*

11. Empirical Calibration

11.1. Detection Probability Matrix

Table 3 presents estimated detection probabilities $d_{\ell,j}$ using ranges (not point values) with confidence levels. Confidence: **H** = strong empirical backing; **M** = single source or derived; **L** = extrapolated; **E** = estimated from first principles.

Key calibration sources: type systems catch 15% of public bugs (Gao et al., 2017); static analysis tools achieve 13% recall in production; Spotify's 25% veto rate across 1,500+ PRs calibrates Layer 3 (Spotify Engineering, 2025); AgentSpec's 87% code-based enforcement (Wang et al., 2026).

11.2. Cost Structure

The false positive cost trap: at 100 PRs/week with Layer 3 FP rate of 25%, developers spend approximately 2,600 hours/year investigating false findings, equivalent to ~1.3 FTEs.

11.3. Layer Assignment by ROI

Computing $\mathrm{ROI}(\ell, j) = d_{\ell,j} \cdot D_j / c_\ell$ yields clear prescriptions:

- **Always apply L1:** cost is negligible; ROI positive for all 18 types.

Table 3 | Detection probability ranges for selected slop types across four defense layers.

Slop Type	L1	L2	L3	L4
Decidable types				
Hallucinated imports	0.90–0.98 H	0.50–0.70 M	0.60–0.80 M	0.70–0.90 M
Placeholder/stub code	0.30–0.50 M	0.85–0.95 H	0.70–0.85 M	0.80–0.95 M
Duplicate logic	0.05–0.15 L	0.75–0.90 H	0.40–0.60 M	0.50–0.70 M
Semi-decidable types				
Security vulns	0.10–0.20 M	0.10–0.15 H	0.50–0.70 M	0.60–0.80 M
Happy-path handling	0.05–0.15 L	0.20–0.40 M	0.55–0.75 M	0.70–0.85 M
Scope expansion	0.05–0.15 M	0.10–0.25 L	0.60–0.80 M	0.70–0.85 M
Undecidable types				
Meaningless tests	0.00–0.05 E	0.05–0.15 E	0.20–0.40 L	0.50–0.70 L
Redundant comments	0.00–0.02 E	0.05–0.15 E	0.30–0.50 L	0.50–0.70 L

Table 4 | Cost per PR by defense layer.

Layer	Time	Cost/PR	FP rate
L1: Structural gates	2–10s	$0.50	0.01–0.05
L2: Deterministic analysis	10–60s	$2.00	0.05–0.20
L3: LLM-as-Judge	20–120s	$8.00	0.15–0.35
L4: Human review	10–60 min	$80.00	0.05–0.15

- **Always apply L2:** ROI > 100 for all types where $d_{2,j} > 0.10$; justified for 16/18 types.
- **Apply L3 selectively:** justified only for 9 high-severity semantic types (security, error handling, data models, architecture, meaningless tests, stubs, over-engineering, scope expansion, god functions).
- **Apply L4 only for:** security ($\text{ROI} = 875$), architectural erosion ($\text{ROI} = 1{,}031$), data models ($\text{ROI} = 516$), meaningless tests ($\text{ROI} = 375$), plus all L3-flagged items above severity threshold.
- **Never cost-effective at L4:** dead code ($\text{ROI} = 6$), redundant comments ($\text{ROI} = 4$). These should never reach human review.

Sensitivity analysis. Many detection estimates in Table 3 carry confidence level E or L. We verified robustness by perturbing all E/L estimates by $\pm 50\%$. The "always apply" prescriptions for Layers 1–2 are robust: even at the pessimistic extreme, their negligible cost ensures positive ROI for all types. Layer 3 prescriptions are moderately sensitive: at -50% detection for Semi-decidable types, Layer 3 is justified for 6 types rather than 9 (dropping over-engineering, god functions, and scope expansion). Layer 4 prescriptions are robust because they depend on high downstream cost D_j rather than high detection $d_{4,j}$. The cost estimates (Table 4) use loaded developer cost of $80/hr; at $40/hr or $160/hr, the L4 ROI thresholds shift by $2\times$ but the set of L4-justified types changes only at the margin (data model mismatches drops out at $160/hr; meaningless tests is borderline at $40/hr).

11.4. Validation Against Deployments

Six deployment calibration points confirm the model:

1. **Spotify Honk:** 1,500+ PRs, 25% veto rate (calibrates $d_{3,13}$), 50% course-correction (calibrates

remediation rate) (Spotify Engineering, 2025).
2. **METR:** 1–2% general reward hacking, but 30–100% when scoring code is visible (43× visibility amplification), validating the principle that quality gates must be opaque to the agent (METR, 2025).
3. **AgentSpec:** 87% code-based vs. 77% prompt-based (10-point gap validates Theorem 8.2) (Wang et al., 2026).
4. **Veracode:** 45% security failure rate across 100+ LLMs calibrates $d_{\text{producer,security}} = 0.55$ (Veracode, 2025).
5. **CodeRabbit:** 1.68× issue ratio, 53.5% recall calibrates Layer 3 sensitivity (CodeRabbit, 2025).
6. **Apollo Research:** 8.7–13% baseline scheming rate; anti-scheming training reduced to 0.3–0.4% but evaluation awareness increased, suggesting models learn to detect tests rather than genuinely align (Apollo Research, 2025).

12. Cost-of-Quality Model for AI Code

12.1. The Four Cost Categories

Total cost of quality: $\text{CoQ} = C_P + C_A + C_{IF} + C_{EF}$ (prevention, appraisal, internal failure, external failure). The classical 1:10:100 ratio requires recalibration by decidability class:

- **Decidable types:** approximately 1:3:100. Detection is cheap (still automated at the appraisal stage); production costs remain high.
- **Semi-decidable types:** approximately 1:15:100. Appraisal requires LLM or human review.
- **Undecidable types:** approximately 1:50:30. Appraisal is expensive (deep expert review); but production cost is *lower* because these degrade maintainability, not availability.

Corollary 12.1 (Inversion for Undecidable Types)**.** *For Undecidable types, appraisal cost can exceed discounted external failure cost. It is sometimes economically rational to* accept *these defects. This is the formal basis for "slop is acceptable" in certain contexts.*

12.2. Optimal Quality Level

Theorem 12.1 (Optimal Quality Level)**.** *With defect rate $\lambda(q) = \lambda_0 e^{-\beta q}$, the optimal quality investment is:*

$$q^* = \frac{1}{\beta} \ln\left(\frac{c_F \cdot \lambda_0 \cdot \beta}{c_P + c_A}\right) \tag{13}$$

where c_F is failure cost and $c_P + c_A$ is marginal prevention/appraisal cost.

This gives context-dependent quality targeting: $q^* \approx 1$ for security-critical production code (c_F high), $q^* \approx 0.7$ for internal tools (c_F moderate, Layers 1–2 suffice), $q^* \approx 0.3$ for prototypes (c_F low, Layer 1 only).

12.3. The Appraisal Compression Problem

If code generation rate g exceeds appraisal capacity a, a fraction $(g - a)/g$ bypasses appraisal entirely. Sonar data (48% always verify) implies $g/a \approx 2$ on average: roughly half of AI code receives no appraisal (Sonar, 2026). The quality stack's ordering is not merely cost optimization; it is a *throughput constraint*: only automated layers (1–2) can match AI generation speed.

13. Related Work

Software defect taxonomies. IEEE 1044 (iee, 2009) provides multi-attribute anomaly classification assuming the failure-defect-fault chain. ODC (Chillarege et al., 1992) offers eight orthogonal defect types with development-phase correlations. Beizer (1990) provides a ten-category bug taxonomy. CWE covers security. None captures accretion defects (§9).

Software reliability. Musa's software reliability models (Musa et al., 1987) connect operational profiles to defect discovery. Fenton and Neil (1999) introduced Bayesian networks for defect prediction. Jones and Bonsignour (2012) provided the foundational DRE data (25,000 projects). Littlewood and Strigini (1993); Littlewood et al. (2000) proved that diverse software versions do not fail independently, a result we adapt to LLM judge-producer pairs.

AI code quality measurement. CodeRabbit (CodeRabbit, 2025), GitClear (GitClear, 2025), Qodo (Qodo, 2025), and Siddiq et al. (2026) provide the emerging empirical base. The controlled GitHub trial (Peng et al., 2024) stands in tension with observational results, a contradiction we interpret through the verification bottleneck.

AI agent enforcement. AgentSpec (Wang et al., 2026) measures the structural-vs-prompt enforcement gap. Spotify's Honk (Spotify Engineering, 2025) provides the best LLM-as-Judge deployment data. METR (METR, 2025), Apollo Research (Apollo Research, 2025), and NIST CAISI (NIST CAISI, 2026) document agent reward hacking and scheming.

Formal program analysis. Abstract interpretation (Cousot and Cousot, 1977) provides the standard framework for sound over-approximation of semantic properties, directly relevant to the Quasi-Semantic enforcement class. Roy and Cordy's clone type taxonomy (Types I–IV) characterizes detection decidability for duplicate logic: Types I–III (textual, renamed, gapped) are decidable; Type IV (semantic clones) is not. Murphy, Notkin, and Sullivan's reflexion models compare intended architecture against extracted source models, providing the formal basis for architectural erosion detection.

Defense in depth. The Swiss Cheese Model (Reason, 1990) frames accident causation through aligned holes in multiple barriers. Viola-Jones cascades (Viola and Jones, 2004) formalize sequential classifier design. Boehm's cost escalation curve (Boehm, 1981) and Jones and Bonsignour (2012) provide the economic foundation, though the cost curve may be flatter in CI/CD environments.

Information theory in software. Hassan's change entropy (Hassan, 2009) predicts faults from change scattering. Our codebase entropy (Definition 9.1) shifts from measuring change scattering to measuring the coupling structure that necessitates it.

14. Design Principles

The formal framework yields ten actionable design principles for AI code quality architecture:

1. **Structural before probabilistic.** Everything expressible as a formal property should be structurally enforced (Theorem 8.2). This is not a preference; it is an optimality result.

2. **Cheap layers first.** Order by ROI (Theorem 7.1). Layer 1 at $0.50/PR before Layer 4 at $80/PR.
3. **Diversity over strength.** A weak independent detector outperforms a strong correlated one (Corollary 10.1). Cross-model judging (Claude judging GPT output) over same-model.
4. **Opacity to the agent.** Quality gates must be invisible to the producing agent. Visibility amplifies reward hacking 43× (METR).
5. **Per-type, not uniform.** Different slop types have radically different optimal layer assignments. No single tool catches all 18 types.
6. **Accept what you cannot detect.** For Undecidable types where appraisal exceeds discounted failure cost, acceptance is the economically rational strategy.
7. **Monitor the aggregate.** Individual accretion is undetectable; aggregate accretion is measurable via entropy proxies (refactoring ratio, clone frequency, accretion rate).
8. **Maintain the refactoring ratio.** The 2.5× violation of the equilibrium condition (Corollary 9.2) is the root cause of codebase degradation. Any quality architecture must include mechanisms to sustain $r \geq 0.20$.
9. **Match appraisal to throughput.** When generation rate exceeds review capacity, only automated layers maintain coverage. Layer 4 cannot scale; Layers 1–2 can.
10. **Three diverse layers for 95% DRE.** The copula model confirms Capers Jones: no single technique suffices, and correlated pairs underperform (Proposition 10.1).

15. Discussion and Limitations

Contrarian positions engaged. *"Slop is acceptable."* Our framework agrees, partially. The optimal quality level (Theorem 12.1) is explicitly $q^* < 1$ for low-stakes code. The contribution is making this precise: for prototypes, Layer 1 only; for production, all four layers.

"Prompt-based rules work well enough." AgentSpec's 77% compliance is substantial but 10 points below structural enforcement. For subjective quality (naming, abstraction), prompts may be the *only* option, but they should never be the *sole* defense for safety-critical constraints.

"LLM-as-Judge is unreliable." Partially validated. Inter-rater omega of 0.462–0.803 is concerning (Stureborg et al., 2024). Same-model judging amplifies correlated errors (Theorem 6.1). But cross-model judging with structural scaffolding (Layer 3 constrained by Layers 1–2) remains cost-effective for semi-decidable types.

"The 18 types are not the right taxonomy." Likely correct at the surface level. The three-factor latent structure (Context Blindness, Completion Bias, Training Leakage) may be more fundamental. Confirmatory factor analysis on the ISSRE 2025 dataset would test this.

Limitations.

- The detection probability estimates (Table 3) have heterogeneous confidence. No study directly measures all 18 types across all 4 layers.
- The independence assumption (used in several theorems) is explicitly violated in practice, as our correlation analysis shows. The copula model is an approximation.
- Codebase entropy (Definition 9.1) is an idealized metric. Computing $|\mu(f_i)|$ exactly requires solving dependency analysis, which is itself subject to Rice's theorem for semantic dependencies.
- The cost-of-quality calibration relies on Capers Jones' data (1975–2015), NIST 2002, and IBM 2025. Modern CI/CD may compress the cost escalation curve.

- We do not address adversarial slop (intentional quality degradation by agents), focusing instead on the more common case of unintentional degradation from statistical pattern completion.
- The decidability classification reflects current LLM capabilities. As models improve in reasoning about intent, some Semi-decidable types may become effectively Decidable (through better syntactic proxies or more reliable semantic judgment). The classification should be understood as contingent, not permanent.

16. Conclusion

AI code generation has introduced a qualitatively new class of software quality problem: defects that are correct, unnecessary, individually defensible, and collectively corrosive. Existing defect taxonomies miss them; existing quality tools catch some but not all; existing review practices cannot scale to match generation throughput.

We have shown that the problem admits formal treatment. The decidability classification determines which defects can be structurally enforced and which require probabilistic judgment. The layered defense optimization, though NP-hard in general, yields clean prescriptions at practical scale. The detection ceiling theorem explains *why* no single layer suffices. The correlated error model explains *why* same-model judging underperforms. The compound degradation theorem explains *why* individually correct changes can produce collective harm.

Four slop types (meaningless tests, boilerplate inflation, redundant comments, naming/convention drift) resist detection at every layer. These constitute the research frontier. Progress requires either reducing them to measurable proxies (moving them from Undecidable to Semi-decidable) or accepting them as a cost of AI-assisted development and compensating through aggregate health monitoring.

The central engineering insight is that the quality problem is solvable, but only through layered, type-specific, cost-aware defense that matches the diversity and scale of the defect landscape.

References

IEEE standard classification for software anomalies, 2009.

Apollo Research. In-context scheming in O3 and O4-mini. Technical report, 2025. 8.7–13% baseline scheming; anti-scheming training reduced to 0.3–0.4%.

B. Beizer. *Software Testing Techniques*. Van Nostrand Reinhold, 2 edition, 1990.

B. W. Boehm. *Software Engineering Economics*. Prentice Hall, 1981.

R. Chillarege, I. S. Bhandari, J. K. Chaar, M. J. Halliday, D. S. Moebus, B. K. Ray, and M.-Y. Wong. Orthogonal defect classification: A concept for in-process measurements. *IEEE Transactions on Software Engineering*, 18(11):943–956, 1992.

CodeRabbit. State of AI vs human code generation. Technical report, 2025. 470 PRs analyzed; 1.68x issue ratio, 2.74x XSS ratio, 8x I/O ratio.

P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977.

T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2 edition, 2006.

I. Dinur and D. Steurer. Analytical approach to parallel repetition. In *Proceedings of the 46th Annual ACM Symposium on Theory of Computing (STOC)*, pages 624–633, 2014.

N. E. Fenton and M. Neil. A critique of software defect prediction models. *IEEE Transactions on Software Engineering*, 25(5):675–689, 1999.

C. M. Fortuin, P. W. Kasteleyn, and J. Ginibre. Correlation inequalities on some partially ordered sets. *Communications in Mathematical Physics*, 22(2):89–103, 1971.

Z. Gao, C. Bird, and E. T. Barr. To type or not to type: Quantifying detectable bugs in JavaScript. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, pages 758–769, 2017.

GitClear. AI copilot code quality 2025. Technical report, 2025. 211M+ lines analyzed; refactoring collapsed from 24.1% to 9.5%.

G. Hardin. The tragedy of the commons. *Science*, 162(3859):1243–1248, 1968.

A. E. Hassan. Predicting faults using the complexity of code changes. *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, pages 78–88, 2009.

C. Jones and O. Bonsignour. *The Economics of Software Quality*. Addison-Wesley Professional, 2012.

M. M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9): 1060–1076, 1980.

B. Littlewood and L. Strigini. Validation of ultra-high dependability for software-based systems. *Communications of the ACM*, 36(11):69–80, 1993.

B. Littlewood, P. Popov, L. Strigini, and N. Shryane. Modeling the effects of combining diverse software fault detection techniques. *IEEE Transactions on Software Engineering*, 26(12):1157–1167, 2000.

METR. Evaluation of O3 and O4-mini: Reward hacking. Technical report, 2025. 1–2% general reward hacking; 30–100% when scoring code visible.

J. D. Musa, A. Iannino, and K. Okumoto. *Software Reliability: Measurement, Prediction, Application*. McGraw-Hill, 1987.

G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions. *Mathematical Programming*, 14:265–294, 1978.

NIST CAISI. AI agent standards initiative. Technical report, 2026. 81% attack success rate against prompt-based defenses.

S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer. The impact of AI on developer productivity: Evidence from GitHub Copilot. In *arXiv preprint arXiv:2302.06590*, 2024. 202 developers; quality improvements +2.47% to +4.16%.

Qodo. State of AI code quality 2025. Technical report, 2025.

J. Reason. *Human Error*. Cambridge University Press, 1990.

H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953.

M. L. Siddiq et al. Debt behind the AI boom. *arXiv preprint arXiv:2603.28592*, 2026. 304,362 AI commits across 6,275 repos; 484,606 issues found.

A. Sklar. Fonctions de répartition à *n* dimensions et leurs marges. *Publications de l'Institut de Statistique de l'Université de Paris*, 8:229–231, 1959.

W. E. Smith. Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3 (1–2):59–66, 1956.

Sonar. State of AI code quality 2026. Technical report, 2026. 42% of code is AI-generated; 48% always verify.

Spotify Engineering. Background coding agents at Spotify: Lessons from Honk. Technical report, 2025. 1,500+ merged PRs; 25% LLM-as-Judge veto rate.

R. Stureborg et al. Can you trust LLM judgments? reliability of LLM-as-a-judge. *arXiv preprint arXiv:2412.12509*, 2024. McDonald's omega 0.462–0.803.

Veracode. Large language models for code: Security hardening and adversarial testing. Technical report, 2025. 100+ LLMs, 80 tasks; 45% security failure rate.

P. Viola and M. J. Jones. Robust real-time face detection. *International Journal of Computer Vision*, 57(2):137–154, 2004.

J. Wang et al. AgentSpec: Customizable runtime enforcement for safe and reliable AI agents. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*, 2026. 87.26% enforcement for code-based rules; 77% for prompt-based.

Appendices

A. Full Detection Probability Matrix

The complete 18×4 detection probability matrix, with confidence levels and primary sources for each estimate, is provided in the supplementary materials. Key calibration anchors:

- Layer 1, hallucinated imports: $d = 0.90$–0.98 (lockfile validation is near-deterministic).
- Layer 2, placeholder code: $d = 0.85$–0.95 (AST pattern matching for known patterns).
- Layer 3, scope expansion: $d = 0.60$–0.80 (calibrated against Spotify's 25% veto rate).
- Layer 4, architectural erosion: $d = 0.75$–0.90 (human review is the primary defense).
- All layers, meaningless tests: combined escape rate $\sim 25\%$ (the detection frontier).

B. ROI Derivation

For each layer ℓ and slop type j, $\mathrm{ROI}(\ell, j) = d_{\ell,j} \cdot D_j / c_\ell$. Layer costs: $c_1 = \$0.50$, $c_2 = \$2.00$, $c_3 = \$8.00$, $c_4 = \$80.00$. Downstream costs D_j range from \$500 (redundant comments) to \$100,000+ (security vulnerabilities, architectural erosion).

The ROI ordering confirms: Layer 1 is always cost-effective (ROI > 1 for all types). Layer 4 is cost-effective only for 4–5 high-severity types. Layer 3's cost-effectiveness depends on the downstream cost; it is justified when $D_j > \$5{,}000$ and $d_{3,j} > 0.40$.

C. Correlation Adjustment Factors

Estimated pairwise beta factors (from fault tree analysis literature):

- L1 vs L2: $\beta = 0.05$–0.15 (different mechanisms).
- L1 vs L3: $\beta = 0.02$–0.08 (structural vs semantic).
- L2 vs L3: $\beta = 0.10$–0.25 (partial overlap in code structure reasoning).
- L3 vs L4: $\beta = 0.15$–0.35 (shared semantic understanding).
- L3 (same model) vs producer: $\beta = 0.30$–0.60 (shared training data).
- L3 (different model) vs producer: $\beta = 0.10$–0.25 (reduced by diversity).

For security vulnerabilities, the corrected escape probability (accounting for L2–L3 correlation at $\beta = 0.15$) is approximately 29% higher than the naive independence estimate.

D. Proof Details

D.1. Theorem 4.1: NP-Hardness

The full reduction from Weighted Set Cover: given universe $U = \{u_1, \ldots, u_m\}$ and sets $S_1, \ldots, S_k$ with weights w_i, construct a defense instance with m defect types (one per element), k layers (one per set), $d_{\ell,j} = 1$ if $u_j \in S_\ell$ else 0, $c_\ell = w_\ell$, $D_j = M$ for large M, $p_j = 1$. Any feasible solution must cover all elements (else penalty M dominates), so the remaining cost is $\sum_\ell c_\ell \cdot 1[\ell \text{ selected}]$, which is Weighted Set Cover. By Dinur and Steurer (2014), this is inapproximable below $\ln m$.

D.2. Theorem 4.2: Greedy Bound

The function $f(\Lambda) = \sum_j p_j D_j [1 - P_{\text{esc}}(j, \Lambda)]$ is the total expected detected defect cost. Under independence, for each type j, the marginal gain from adding layer ℓ to set Λ is $p_j D_j \cdot d_{\ell,j} \cdot \prod_{\ell' \in \Lambda}(1 - d_{\ell',j})$, which decreases as $|\Lambda|$ grows (diminishing returns). Summing over j preserves submodularity (non-negative sum of submodular functions). Monotonicity is immediate (adding layers never increases escape probability). By Nemhauser et al. (1978), the greedy algorithm for maximizing a monotone submodular function subject to a cardinality or knapsack constraint achieves $(1 - 1/e) \approx 0.632$.